



# AI Video Production Specialist

A Digital FTE that turns a story brief into a publish-ready, merged video — autonomously.



Autonomous orchestrator



Veo 3 / Runway



ElevenLabs voice



Sacred Guard



Face-Lock



**Facts only.** Every box below is grounded in the repo source ( `src/` , `config/` , `specs/ai-video-fte/spec.md` ). Where the *current code* and the *intended secure design* differ (API-key storage), both are shown.

## 1 · What this FTE is

### Role

**AI Video Production Specialist** — end-to-end video production from story concept to final merged video.

**Autonomy:** accepts a brief, then *independently* plans shots, selects models, generates, verifies & merges.

spec §2

### Escalates only on



Sacred Guard policy violation



Cost Guard budget overrun

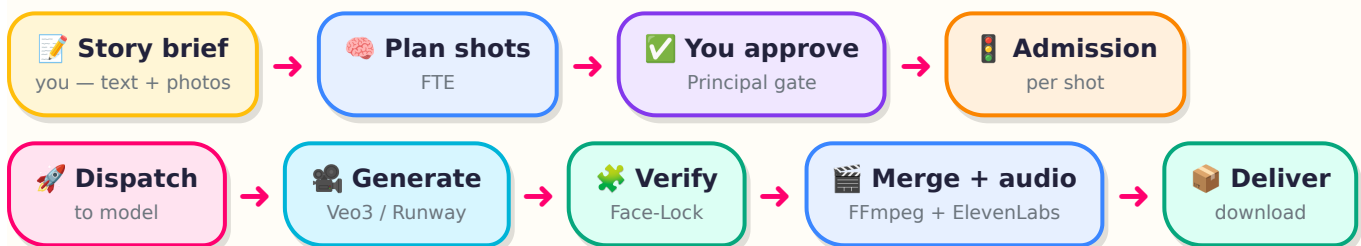


Unrecoverable technical failure

**Non-goals:** no creative direction, no hosting/streaming (delivers downloads), no accounts/billing.

spec §2

## 2 · One-screen mental model



Runs as one service, two ports + background workers:

| Surface                                                           | Port | Set by                           | Source        |
|-------------------------------------------------------------------|------|----------------------------------|---------------|
| API server (stories, admission, router, dispatch, merger, health) | 3000 | API_PORT                         | src/server.ts |
| Metrics server (Prometheus <code>/metrics</code> )                | 9090 | config.observability.metricsPort | src/server.ts |

### 3 · What you must supply — API keys

The FTE can't render pixels or audio itself. It **orchestrates external providers**, and those calls need API keys.

#### Video keys — mandatory for real output

| Provider                       | Env var        | Cost/s              |
|--------------------------------|----------------|---------------------|
| <b>Veo 3</b> (ultra-realistic) | VEO_API_KEY    | \$0 low / \$0.05 hi |
| Runway Gen-3 (fallback)        | RUNWAY_API_KEY | \$0.08              |
| Luma Ray 2                     | adapter        | \$0.03              |

`src/shared/config.ts` ,  
`config/development.yaml`

#### Audio / voice key — mandatory if story has audio

**ElevenLabs** — text-to-speech voice-over *and* one of the background-music sources.

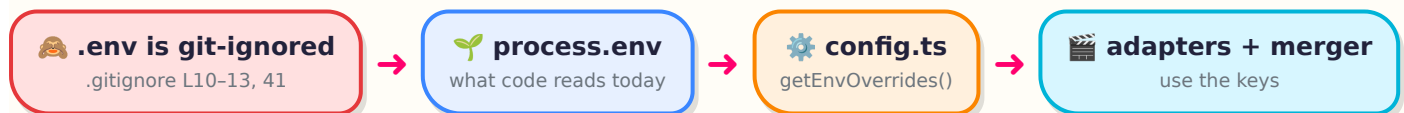
`ELEVENLABS_API_KEY`

`merger.generateTTSAudio()` **throws** if it's missing. A silent video needs no key; any voice/music does.

 **Music has 3 sources** ( `MusicConfig.source` ): `royalty_free` `elevenlabs` `custom` — the built-in **royalty-free library** covers background music with no per-track key; user may override with own assets.

`src/merger/merger.ts` , `types.ts:MusicConfig` ,  
spec FR-028/CL-022

### 3.3 · Where keys live — `.env` , `.gitignore` & Vault



**+ How to add keys now** — create a local `.env` (stays out of git):

```

VEO_API_KEY=veo_live_xxx      # video generation
RUNWAY_API_KEY=rw_xxx        # fallback video
ELEVENLABS_API_KEY=el_xxx    # voice + music (needed if story has audio)
VAULT_ADDR=http://127.0.0.1:8200 # Vault connection only
VAULT_TOKEN=hvs.xxx
VAULT_TRANSIT_KEY=biometric-encryption-dev
  
```

 **Honest caveat (is vs should-be).** Today provider keys flow through `process.env` ( `config.ts:getEnvOverrides` ). The *documented secure target* ( `CLAUDE.md` \$16 + `src/shared/vault.ts` , AES-256-GCM envelope encryption) is to keep provider keys **in Vault** and put only Vault's own connection creds in `.env` . Wiring `config.ts` to fetch provider keys from Vault at runtime is a Build task — not something current code already does.

### 4 · How it orchestrates its workers

Event-driven: every state change is written to **PostgreSQL** (durable, immutable) *and* published to a **Redis Stream** (real-time). Workers consume the streams.

4 durable streams

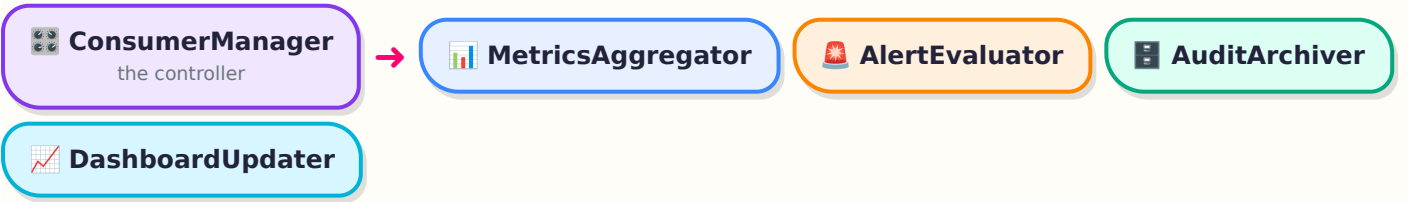
story\_commands   story\_events   webhook\_ingress  
job\_status  
src/shared/redis.ts   STREAMS

8 consumer groups

command-handler   event-processor   webhook-handler  
job-monitor   metrics-aggregator   alert-evaluator  
audit-archiver   dashboard-updater  
at-least-once · XACK on success · XAUTOCLAIM recovers stalled work

AI Video Production Specialist · Workflow Guide · API keys & secrets

The workers the FTE controls — and how it controls them

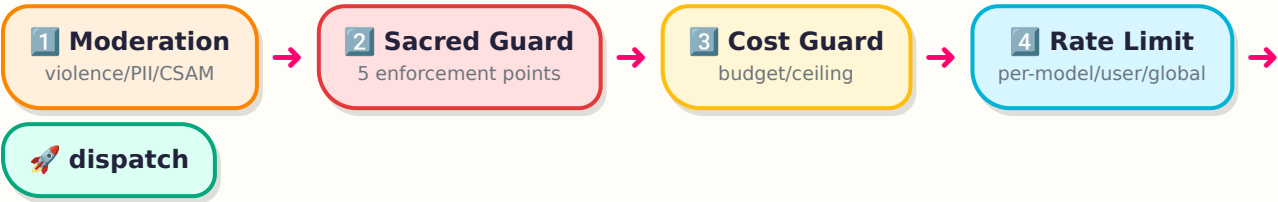


| Control lever       | What happens                                                                                                 | Source                            |
|---------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------|
| ▶ Create / start    | Instantiates each worker w/ unique consumer name; a failed start aborts startup                              | ConsumerManager.start()           |
| ♥ Health-check      | Every <b>30s</b> : marks <i>degraded</i> if >10% failures after 100 msgs; writes <code>health_metrics</code> | startHealthChecks()               |
| ↺ Restart           | Stops+starts a single worker without touching the others                                                     | restartConsumer(name)             |
| ● Graceful shutdown | Races each stop against a 30s timeout; wired to SIGTERM/SIGINT                                               | stop() /<br>setupSignalHandlers() |

🧠 **The pipeline brain = two state machines** ( `src/shared/events.ts` ): `storyStateMachine` & `shotStateMachine` . Every transition runs guards + side-effects, then emits an immutable event to Postgres **and** Redis. The `generating→merging` transition auto-fires the cross-shot Face-Lock report.

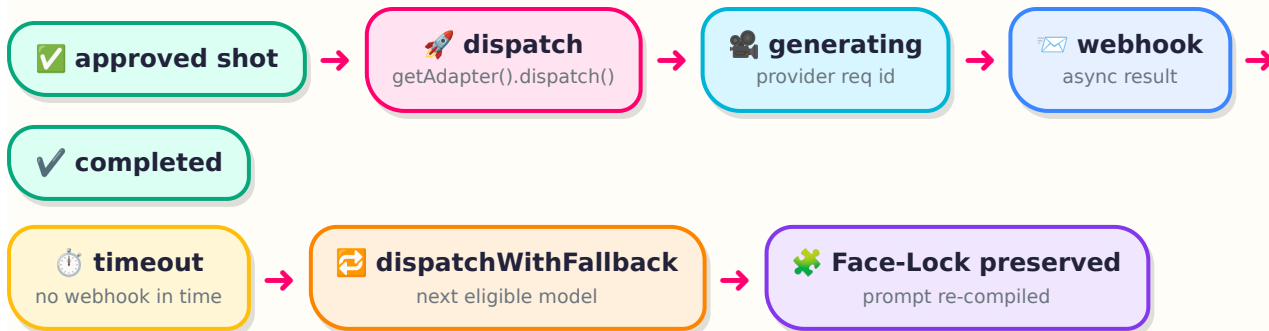
5 · The complete workflow 🔗

5.1 · Admission — fixed, immutable order



src/admission/admissionController.ts · spec FR-009. A block flips the story to the matching `paused_*` state → Principal decides.

## 5.2 · Dispatch → generate → fallback



🛡️ `isShotDispatched()` prevents duplicate dispatch. ⌚ `timeout→dispatched` means a lost callback never strands a shot or double-charges. 📄 `src/dispatch/shotDispatcher.ts`

### 5.3 · Face-Lock 🧩

Reference image conditioned per-model → generated frames compared (cosine similarity) vs registry → below threshold auto-regenerates (default **2 retries**) → cross-shot report on merge.

📄 `faceLockVerification.ts`, `promptCompiler.ts`

### 5.4 · Merge · audio · subtitles 🎬

`mergeStoryShots()` → FFmpeg `libx264 crf23 aac 128k +faststart` · native audio preserved if the model emits it · voice via ElevenLabs · music from royalty-free library / ElevenLabs / custom · subtitles SRT (default)/VTT/ASS ·

`generateDeliveryPackage()` → signed URL, 7-day TTL.

📄 `src/merger/merger.ts` · `getMusicTrack()` stub today

AI Video Production Specialist · Workflow Guide · orchestration & workflow

## 6 · Transitions — who generates vs who applies 🗂️

**Key fact:** transitions are **NOT AI-generated**. They are a fixed catalog of pre-defined FFmpeg `xfade` presets. The FTE picks one; FFmpeg renders it.

### 🏗️ GENERATE / DEFINE `src/merger/transitionSystem.ts`

21 hardcoded presets: crossfade (default 0.5s→'fade'), slide, slideL/R/U/D, zoom/In/Out, wipe/R/U/D, circleOpen/Close, pixelize, hISlice, vISlice, distance, smooth  
`buildTransitionFilter()` → xfade string

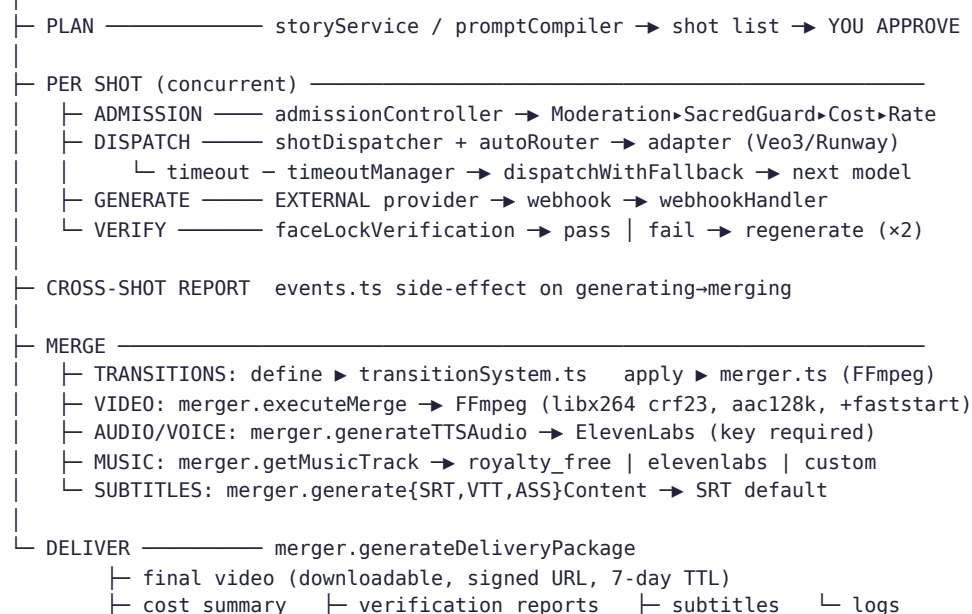
### 🎬 APPLY / RENDER `src/merger/merger.ts`

→ `mergeStoryShots()` picks `options.transition || config.merger.defaultTransition`  
`executeMerge()` spawns FFmpeg → `xfade` runs between each adjacent pair of shots

| Aspect            | Fact                                                                                                     |
|-------------------|----------------------------------------------------------------------------------------------------------|
| Default           | <code>crossfade , 0.5s ( config/development.yaml merger.defaultTransition; preset → ffmpeg fade )</code> |
| Per-shot override | <code>mergeTransitionConfigs(global, shot)</code> — shot wins, else global, else crossfade 0.5s          |
| Offset math       | <code>offset = max(0, prevDuration - transitionDuration)</code>                                          |
| One line          | <code>transitionSystem.ts</code> defines the filter · <code>merger.ts</code> applies it via FFmpeg       |

## 7 · Result-generation tree 🌳

STORY BRIEF (you)



## 8 · How to use it 🗺️

1. Set keys (\$3) → 2. `POST /stories` → 3. Review plan → 4. Approve/revise → 5. Watch `:9090/metrics` + dashboards → 6. Handle `paused_*` escalations → 7. Collect delivery → 8. Optional partial re-gen (`merger.partialRegenerate`).

## 9 · How it behaves 🎯

Sacred match → blocked, no gen. Over budget → pauses before dispatch. Model fails → auto fallback. Lost callback → auto-recovery, no double charge. Same character → consistent every shot. Re-gen 2 shots → only those redone, re-merged.

📄 spec SC-002...SC-008

📦 **What you get:** a merged, publish-ready MP4 ( `+faststart` ) at your resolution/aspect ratio + subtitles + cost summary + Face-Lock/cross-shot reports — behind a 7-day signed URL.

AI Video Production Specialist · Workflow Guide · Verify every claim: see §10 of `docs/workflow-guide.md` . Behaviour-level guide; code is the build output and the source of truth.